

Instituto Superior de Engenharia

Politécnico de Coimbra

Instituto Superior de Engenharia de Coimbra

Relatório Técnico do Projeto AWS

PollNow

Aplicação Serverless para Criação, Partilha e Votação em Sondagens Online

Autores: Ricardo Rodrigues e Ruben Ferreiro

Frontend: https://pollnow.netlify.app/

Repositório: https://github.com/RicardoDR14/pollnow-aws

Data: 09/06/2026

Índice

1. Resumo executivo

2. Objetivos do projeto

3. Tecnologias utilizadas

4. Arquitetura geral

5. Estrutura do repositório

6. Serviços AWS utilizados

7. Modelo de dados

8. Backend e rotas da API

9. Fluxos principais de funcionamento

10. Configuração e deployment

11. Validação e testes

12. Decisões técnicas e justificação

13. Conclusão

1. Resumo executivo

O PollNow é uma aplicação web serverless para criação, partilha e votação em sondagens online. A aplicação permite que utilizadores autenticados criem e administrem as suas próprias sondagens, enquanto participantes externos podem votar através de páginas públicas, links diretos ou QR Codes.

O projeto foi desenvolvido com React no frontend e uma arquitetura AWS serverless no backend. A solução integra API Gateway, AWS Lambda, DynamoDB, S3, SNS, EventBridge e CloudWatch. O resultado é uma aplicação funcional que demonstra vários conceitos de cloud: computação sem servidor, armazenamento persistente, armazenamento de objetos, notificações, agendamento automático e separação entre fluxos privados e públicos.

2. Objetivos do projeto

- Desenvolver uma aplicação prática e demonstrável para gestão de sondagens online.
- Aplicar uma arquitetura serverless usando múltiplos serviços AWS.
- Separar claramente a gestão privada das sondagens e o acesso público de votação.
- Garantir persistência de utilizadores, sondagens e votos.
- Adicionar funcionalidades cloud relevantes: upload para S3, notificações via SNS e processamento automático via EventBridge.
- Produzir documentação técnica que explique a estrutura, configuração e funcionamento do sistema.

3. Tecnologias utilizadas

Camada	Tecnologia	Utilização
Frontend	React	Interface web, dashboard, páginas de login/registo, votação, resultados e partilha.
Frontend hosting	Netlify	Publicação da aplicação React e suporte a rotas SPA.
Comunicação HTTP	Axios	Chamadas do frontend para a API.
Navegação	React Router	Gestão das rotas internas: /, /login, /vote, /results e /share.
Gráficos	Recharts	Visualização dos resultados das sondagens.
QR Code	qrcode.react	Geração dos QR Codes de partilha.
Backend	AWS Lambda / Node.js	Execução da lógica backend sem servidor dedicado.
API	Amazon API Gateway	Exposição dos endpoints HTTP consumidos pelo frontend.
Base de dados	Amazon DynamoDB	Armazenamento de utilizadores, sondagens e votos.
Ficheiros	Amazon S3	Armazenamento de imagens e ficheiros CSV de resultados.
Notificações	Amazon SNS	Envio de emails administrativos e resultados por proprietário.
Agendamento	Amazon EventBridge	Execução periódica do processamento de sondagens expiradas.

Monitorização	Amazon CloudWatch	Logs e validação das execuções das Lambdas.

4. Arquitetura geral

A arquitetura do PollNow segue o modelo serverless. O utilizador acede ao frontend React alojado no Netlify. O frontend comunica com o API Gateway através de HTTP. O API Gateway encaminha cada rota para uma função Lambda específica. As Lambdas usam DynamoDB para dados, S3 para ficheiros e SNS para notificações. Em paralelo, uma regra EventBridge executa periodicamente uma Lambda responsável por processar sondagens expiradas.

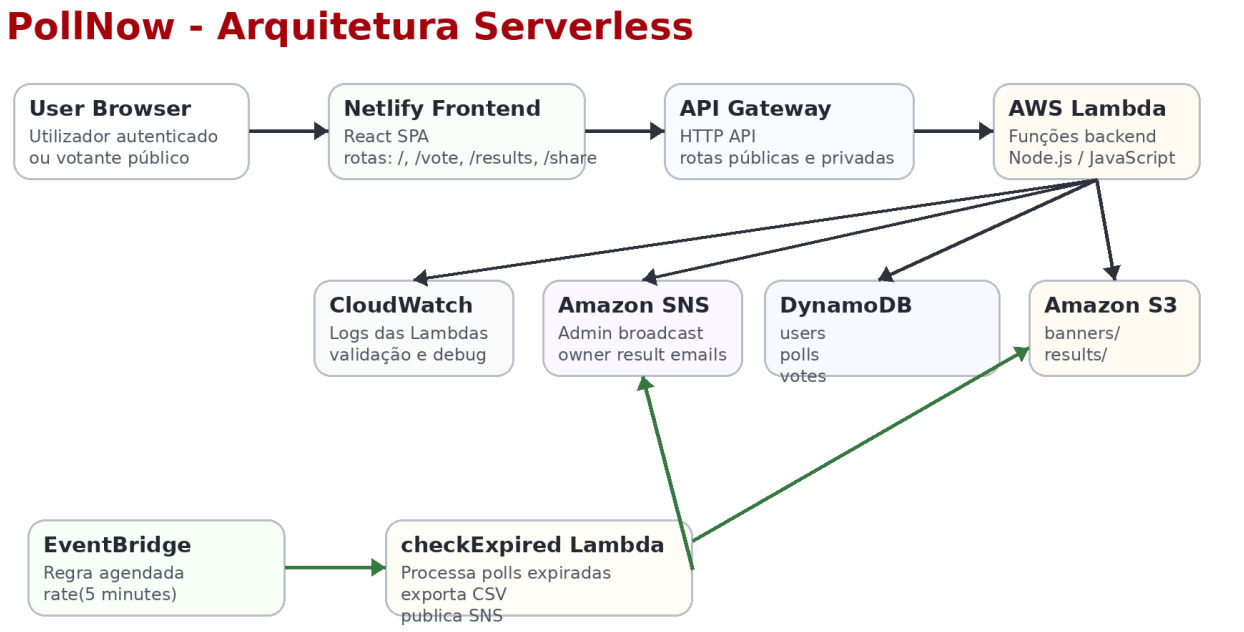


Figura 1 - Arquitetura geral da aplicação PollNow.

5. Estrutura do repositório

O repositório está dividido em três áreas principais: backend, frontend e documentação.

Pasta	Descrição
backend/lambdas	Código das funções AWS Lambda. Cada função representa uma responsabilidade específica da API.
frontend	Aplicação React publicada no Netlify.
frontend/src/pages	Páginas principais: dashboard, login, registo, votação, resultados e partilha.
frontend/src/components	Componentes reutilizáveis, como notificações/toasts.
docs	Documentação final do projeto, arquitetura,

	deployment, testes e limitações.
docs/archive	Histórico das fases de implementação e abordagens antigas.

6. Serviços AWS utilizados

Serviço	Responsabilidade no sistema	Justificação
API Gateway	Expõe as rotas HTTP da aplicação.	Permite ligar o frontend às Lambdas de forma simples e serverless.
AWS Lambda	Executa operações de backend: login, polls, votos, resultados, notificações.	Evita gestão de servidores e permite separar responsabilidades por função.
DynamoDB	Guarda users, polls e votes.	Base NoSQL gerida, adequada a acessos por chave e dados flexíveis.
Amazon S3	Guarda imagens das polls e CSVs de resultados.	Separação correta entre dados estruturados e objetos/ficheiros.
Amazon SNS	Envia emails de ações administrativas e resultados por owner.	Serviço gerido de pub/sub adequado a notificações por email.
EventBridge	Executa o processamento periódico de polls expiradas.	Automatiza tarefas sem depender de interação manual.
CloudWatch	Regista logs das Lambdas.	Facilita validação, debugging e acompanhamento de erros.

7. Modelo de dados

A persistência é feita em DynamoDB através de três tabelas principais.

Tabela	Chave primária	Atributos principais
users	userId	username, email, passwordHash, createdAt, notificationSubArn, notificationsEnabled
polls	pollId	ownerId, ownerEmail, title, description, options, closesAt, status, imageUrl, imageKey, imageContentType, createdAt
votes	voteId	pollId, voterId, option, createdAt, updatedAt

A tabela users identifica os utilizadores e guarda o estado das notificações. A tabela polls guarda os metadados das sondagens e o estado atual: open, closed ou notified. A tabela votes guarda os votos anónimos. O voterId é gerado e guardado no browser, permitindo alterar o voto enquanto a sondagem estiver aberta.

8. Backend e rotas da API

O backend é composto por funções Lambda especializadas. Esta divisão facilita a leitura, manutenção e demonstração das responsabilidades de cada endpoint.

Rota	Lambda	Acesso	Função
POST /register	registerUser	Público	Criar utilizador.
POST /login	loginUser	Público	Autenticar utilizador e devolver estado de notificações.
GET /polls	listPolls	Privado	Listar sondagens do utilizador autenticado.
POST /polls	createPoll	Privado	Criar sondagem e fazer upload opcional de imagem.
GET /polls/{pollId}	getPoll	Público	Obter dados públicos da sondagem.
PUT /polls/{pollId}	updatePoll	Privado	Editar sondagem do próprio utilizador.
DELETE /polls/{pollId}	deletePoll	Privado	Eliminar sondagem e votos associados.
PATCH /polls/{pollId}/close	closePoll	Privado	Fechar sondagem manualmente.
POST /polls/{pollId}/vote	castVote	Público	Criar ou atualizar voto anónimo.
GET /polls/{pollId}/results	getResults	Público	Calcular e devolver resultados.
POST /notifications	toggleNotifications	Privado	Ativar/desativar notificações do owner.
GET /notifications	getNotificationStatus	Privado	Consultar estado da subscrição.
EventBridge trigger	checkExpired	Agendado	Processar polls expiradas, exportar CSV e notificar owner.

9. Fluxos principais de funcionamento



Figura 2 - Fluxos principais do sistema PollNow.

9.1 Criação de sondagem

1. O utilizador autenticado preenche título, descrição, opções, data de fecho e imagem opcional.
2. O frontend envia a requisição POST /polls com o header x-user-id.
3. A Lambda createPoll valida os dados, grava a poll no DynamoDB e, se existir imagem, envia-a para S3.
4. A aplicação abre a página de partilha com QR Code.

9.2 Votação pública

5. O participante acede a /vote/{pollId}, normalmente por link ou QR Code.
6. A página consulta getPoll para obter título, opções, imagem e estado.
7. Ao votar, castVote grava ou atualiza o voto na tabela votes.
8. Depois do voto, o utilizador é redirecionado para a página de resultados.

9.3 Resultados

9. A página /results/{pollId} chama getResults.
10. A Lambda agrega votos por opção e calcula totais e percentagens.
11. O frontend apresenta gráfico e lista de resultados.

9.4 Processamento automático

12. EventBridge chama checkExpired a cada 5 minutos.
13. checkExpired procura polls expiradas ainda não notificadas.
14. A função calcula resultados finais, exporta CSV para S3 e publica mensagem SNS com ownerId.
15. O SNS entrega o email apenas ao owner subscrito com filter policy correspondente.

10. Configuração e deployment

10.1 DynamoDB

Devem existir as tabelas users, polls e votes, com chaves primárias userId, pollId e voteId. As Lambdas usam estes nomes através de variáveis de ambiente.

10.2 S3

O bucket S3 guarda objetos em dois prefixos: banners/ para imagens de sondagens e results/ para CSVs finais. As imagens têm de ser legíveis pelo frontend; numa solução de produção seria recomendável usar CloudFront ou URLs pré-assinadas.

10.3 SNS

O tópico pollnow-notifications é usado para notificações administrativas e para emails de resultados. As subscrições por owner usam filter policy baseada em ownerId, permitindo que o utilizador receba apenas os resultados das suas próprias sondagens.

10.4 EventBridge

A regra pollnow-check-expired executa checkExpired com a frequência rate(5 minutes). Esta automação permite fechar/processar sondagens sem intervenção manual.

10.5 Variáveis de ambiente

Variável	Utilização
USERS_TABLE	Nome da tabela de utilizadores.
POLLS_TABLE	Nome da tabela de sondagens.
VOTES_TABLE	Nome da tabela de votos.
S3_BUCKET	Bucket usado para imagens e CSVs.
SNS_TOPIC_ARN	ARN do tópico SNS.
SNS_REGION	Região usada pelo SNS, normalmente us-east-1.
FRONTEND_URL	URL pública do frontend Netlify.

10.6 Netlify

O Netlify aloja a aplicação React e usa a variável REACT_APP_API_URL para apontar para o API Gateway. O ficheiro _redirects garante que rotas SPA como /vote/{pollId}, /results/{pollId} e /share/{pollId} funcionam por acesso direto.

11. Validação e testes

A validação funcional deve confirmar os principais fluxos da aplicação.

- Registo, login e logout.
- Separação das polls por utilizador.
- Criação de poll com e sem imagem.
- Upload e visualização da imagem nas páginas públicas.
- Edição, fecho e eliminação de poll.
- Voto anónimo e alteração de voto enquanto a poll está aberta.
- Bloqueio de voto em poll fechada.
- Visualização correta dos resultados.
- QR Code a apontar apenas para a página pública de votação.

- Subscrição e receção de notificações SNS.
- Processamento automático via EventBridge/checkExpired.
- Existência de CSV final no S3.

Além dos testes funcionais, CloudWatch deve ser usado para confirmar que as Lambdas executam sem erros e que checkExpired publica as mensagens esperadas.

12. Decisões técnicas e justificação

Decisão	Justificação técnica
Arquitetura serverless	Reduz necessidade de servidores, facilita integração com serviços AWS e ajusta-se ao âmbito académico.
Uma Lambda por responsabilidade	Melhora organização, rastreabilidade e manutenção.
DynamoDB em vez de base relacional	Estrutura simples por documentos/itens, adequada a users, polls e votes.
S3 para imagens/CSVs	Evita guardar ficheiros na base de dados e usa armazenamento adequado a objetos.
SNS com filter policy	Permite usar um tópico partilhado e entregar resultados apenas ao proprietário correto.
EventBridge para expiração	Automatiza processamento temporal sem depender do utilizador ou frontend.
QR Code para partilha	Facilita demonstração e acesso público seguro apenas à votação.

13. Conclusão

O PollNow cumpre o objetivo de demonstrar uma aplicação prática baseada em arquitetura serverless AWS. A solução integra frontend React, backend em AWS Lambda, persistência em DynamoDB, armazenamento em S3, notificações SNS, automação EventBridge e monitorização com CloudWatch.

O projeto demonstra separação entre frontend e backend, diferenciação entre fluxos públicos e privados, uso justificado de serviços AWS e implementação de funcionalidades completas: autenticação, gestão de sondagens, votação anónima, QR Code, resultados, imagens, notificações e processamento automático. Apesar das limitações identificadas, a aplicação é funcional, demonstrável e extensível para uma versão de produção.